

Shor's Algorithm

Contents

1	Introduction	3
2	Algorithm	3
2.1	What is an Algorithm?	3
2.2	Why Do Better Algorithms Matter?	3
2.3	Classical Algorithms vs Quantum Algorithms	4
2.4	The Black Box Approach	4
3	Shor's Algorithm	4
3.1	The Problem of Factorization	4
3.2	Impact	5
3.3	Enter Shor's Algorithm	5
3.4	How Does the Period Help Us?	7
3.5	The Complete Algorithm	7
4	Quantum Fourier Transform	9
4.1	A Fourier Transform Intuition	9
4.2	The Simplest Example	9
4.3	How Does QFT Reveal a Period?	10
4.4	The QFT Circuit	10
4.5	QFT Inside Shor's Algorithm	11
4.6	Qiskit Implementation of QFT	11
5	Shor's Algorithm Step by Step	11
6	Implementing Shor's Algorithm	12
6.1	Choosing the Number to Factor	13
6.2	Constructing the Periodic Function	13
6.3	Preparing the Quantum State	13
6.4	Applying the Quantum Fourier Transform	14
6.5	Extracting the Period	14
6.6	Recovering the Factors	15
6.7	Complete Workflow Summary	15
7	Limitations of Current Quantum Hardware	16
8	Final Remarks	16
9	Further Reading	16

1 Introduction

In this section we are going to talk about one of the most famous algorithms in all of quantum computing.

The only prerequisite for this topic is a basic understanding of how quantum computing works. I will try to build the explanation from the ground up and introduce new concepts whenever they are needed.

Whenever we study a topic, there are usually two layers of understanding:

- The first layer is developing an intuition for what is happening and why the idea is interesting.
- The second layer is understanding the mathematics behind the idea and why the algorithm behaves the way it does.

Both layers are important. The first is important for us to understand the applications of the algorithm, while the second helps us understand the mechanism behind it and think about variations.

Now, Shor's Algorithm is a bit special. Unlike some other quantum algorithms, it relies heavily on mathematical ideas from both quantum computing and number theory. Because of this, some parts of the chapter may feel a little challenging at first. This is completely normal. Focus more on building an intuitive feel first, as the mathematics will become easier when you revisit this topic or as you spend time with quantum computing in general.

With that said, let's get started.

2 Algorithm

2.1 What is an Algorithm?

Before talking about quantum algorithms, let's first understand what an algorithm actually is.

An algorithm is simply a set of instructions that tells us how to solve a standard problem. It is more like a recipe; just as a recipe tells us the required steps to cook a dish, an algorithm tells a computer the steps required to solve a task.

For example, suppose I ask you to find the largest number in a list. One way to do this would be to look at each number one by one and keep track of the largest value seen so far. The sequence of steps that you follow to solve this problem is an algorithm.

The interesting thing is that there can be multiple algorithms for solving the same problem. Some algorithms may take more time, while others may solve the problem much faster.

This brings us to an important question: if different algorithms can solve the same problem, how do we decide which one is better?

2.2 Why Do Better Algorithms Matter?

Let's say that I give you a very fat dictionary that contains the meaning of all the names in India and ask you to find the meaning of the name "Shubham".

One approach would be to start from the first page and check every name until you find the one you are looking for. This works, but it is not very efficient.

A better approach would be to take advantage of the fact that the names are arranged alphabetically (since it's a dictionary). So we will open the dictionary and see whether the word that we are seeing is greater or less than the desired word and accordingly we will shift the pages (assuming you know how to read a dictionary). So this is clearly a better approach to solving the same problem.

This is why algorithms are important. Sometimes the limitation is not the computer itself, but the algorithm being used. A better algorithm can often save a huge amount of time and computational effort.

2.3 Classical Algorithms vs Quantum Algorithms

Now that we know what an algorithm is, let's talk about quantum algorithms.

A quantum algorithm is still an algorithm. It takes some input, performs a sequence of operations, and produces an output. The main difference is that instead of running on a classical computer, it runs on a quantum computer.

Quantum algorithms make use of quantum mechanical properties such as superposition, interference, and entanglement. These properties allow quantum computers to process information in ways that are not possible on classical computers.

This does not mean that quantum algorithms are better at everything. In fact, for many problems classical algorithms are still the better choice.

However, for certain specific problems, quantum algorithms can outperform the best known classical algorithms. Shor's Algorithm is one of the most famous examples of this.

2.4 The Black Box Approach

Whenever I learn a new algorithm, I usually follow what I like to call the *black box approach*.

The idea is simple. Before worrying about how an algorithm works internally, I first try to understand what problem it is solving and what output it produces for a given input.

In other words, I treat the algorithm like a black box. I put something into the box, observe what comes out, and try to understand why this transformation is useful.

Once we are comfortable with what the algorithm is trying to achieve, we can slowly open the black box and study the individual components that make it work.

I find this approach much less overwhelming than trying to understand every mathematical detail from the very beginning.

3 Shor's Algorithm

3.1 The Problem of Factorization

Now that we have a framework on how to approach this algorithm, let's start by talking about what problem Shor's Algorithm is trying to solve.

Suppose I ask you to multiply two numbers:

$$3 \times 7 = 21.$$

Most of us can do this calculation within a few seconds.

Now let's reverse the problem. Suppose I give you the number

$$21$$

and ask you to find the two numbers that were multiplied together to obtain it.

In this case, the answer would be

$$21 = 3 \times 7.$$

This process of breaking a number into the numbers that were multiplied to obtain it is known as *factorization*.

For small numbers, factorization is quite easy. However, as the numbers become larger and larger, the problem becomes significantly harder.

3.2 Impact

I think that you might be disappointed to learn that the big-shot Shor's Algorithm is used to find factors of integers.

The surprising part is that a large portion of modern cryptography relies on this very problem.

Multiplying two large prime numbers together is relatively easy, even for a classical computer. However, if I only give you the final product and ask you to recover the original prime numbers, the problem suddenly becomes much harder.

In fact, the security of many cryptographic systems is built on the assumption that this task is computationally infeasible for classical computers.

In other words, a problem that looks like a simple exercise from a mathematics textbook is actually helping protect digital communication all around the world.

So, Shor's Algorithm showed that a sufficiently powerful quantum computer could factor large integers dramatically faster than the best known classical algorithms.

Believe it or not, this result was so groundbreaking that it made people seriously rethink the future of cryptography. A problem that was considered extremely difficult for classical computers suddenly looked much less secure in the presence of quantum computers.

This is one of the major reasons why quantum computing started receiving so much attention from researchers around the world.

3.3 Enter Shor's Algorithm

The key idea behind Shor's Algorithm is surprisingly simple:

Instead of trying to find the factors directly, we first solve a completely different problem known as *period finding*.

Many times a simple problem helps us solve a bigger problem.

Let's look at a simple example. Consider the function

$$f(x) = 2^x \bmod 15$$

If you are unfamiliar with modular arithmetic, don't worry. For now, all you need to know is that "(mod15)" means that we only keep the remainder after division by 15.

Let's compute a few values:

$$\begin{aligned} f(0) &= 1 \\ f(1) &= 2 \\ f(2) &= 4 \\ f(3) &= 8 \\ f(4) &= 1 \end{aligned}$$

Do you see some pattern here? After four steps, the sequence starts repeating itself:

$$1, 2, 4, 8, 1, 2, 4, 8, \dots$$

The number of steps after which a sequence starts repeating is called its *period*. In this example, the period is

$$r = 4$$

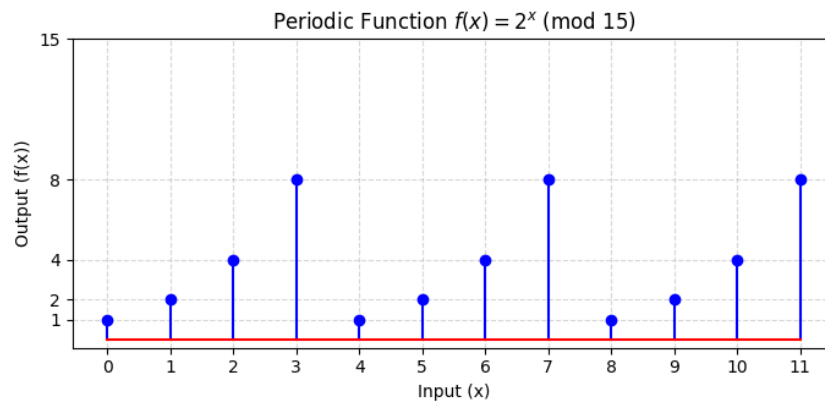


Figure 1: Visualizing the repeating cycle ($r = 4$) of the modular function.

So the thing is, if we can find this period efficiently, then we can use it to recover the factors of the original number. So basically we have reduced the problem of finding the factors to a simple problem of finding periods.

Unfortunately, finding the period of such functions can itself be very difficult for classical computers when the numbers become large. This is where quantum computers come into the picture.

Using a quantum technique known as the Quantum Fourier Transform (QFT), a quantum computer can determine this period much more efficiently than the best known classical methods.

So if we look at Shor's Algorithm from a black-box perspective, it looks something like this:

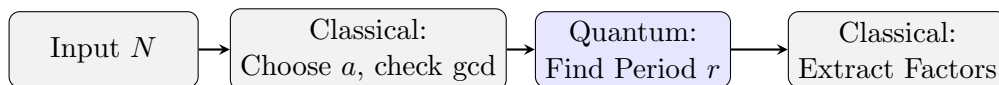


Figure 2: The hybrid classical-quantum workflow of Shor's Algorithm.

Shor's magic lies in the middle step. Once the period is known, the remaining calculations can be performed using ordinary classical computation.

3.4 How Does the Period Help Us?

So how exactly does finding the period help us find factors?

Let's continue with the example we used earlier. We considered the function

$$f(x) = 2^x \bmod 15$$

and found that its period is $r = 4$. Now let's look at the following quantity:

$$a^{r/2}$$

Substituting $a = 2$ and $r = 4$, we get

$$2^{4/2} = 2^2 = 4.$$

Now consider

$$a^{r/2} - 1 \quad \text{and} \quad a^{r/2} + 1.$$

For our example, these become

$$4 - 1 = 3 \quad \text{and} \quad 4 + 1 = 5.$$

Notice anything familiar?

$$15 = 3 \times 5.$$

We have successfully recovered the factors of 15!

Now, before things become somewhat misleading, I want to state that there are a few mathematical conditions that must be satisfied for this trick to work. We will discuss them shortly.

The important thing to understand right now is that once we know the period, we can use a small amount of classical computation to recover the factors.

This is why period finding is the heart of Shor's Algorithm. The difficult part of the algorithm is finding the period efficiently. Once that information is available, obtaining the factors becomes much easier.

This is also the reason why the quantum computer is only responsible for the period-finding step, while the remaining calculations can be performed on a classical computer. (From this you can also get a feel for how quantum algorithms use classical computation wherever necessary.)

3.5 The Complete Algorithm

Now that we understand the main idea behind Shor's Algorithm, let's put all the pieces together. Suppose we are given a large integer N and our goal is to find its factors.

The first thing we do is choose a random integer a such that

$$1 < a < N.$$

At this stage, a is simply a helper number that we will use to construct a periodic function.

Before doing anything quantum, we perform a quick classical check. We compute

$$\gcd(a, N).$$

If this value is greater than 1, then we have already found a non-trivial factor of N , and the algorithm can stop immediately. For this method to continue into the quantum part, we need a and N to be *coprime* (meaning they share no common factors other than 1).

If this does not happen, we move to the next step and define the function

$$f(x) = a^x \bmod N.$$

You might remember from the previous section that functions of this form eventually start repeating themselves. The number of steps after which they repeat is called the period.

Now comes the most important part of the entire algorithm. We need to determine the period r of the function. Classically, finding this period can be extremely difficult when N becomes large. This is exactly where the quantum computer helps us. Using quantum computation, we can find this period much more efficiently than the best known classical methods.

Once the period r has been found, the hard part is essentially over. However, not every period leads to useful information. For the next step to work, the period must be even. If r is odd, we cannot divide it neatly by 2 to get a whole number exponent, so we simply choose a different value of a and repeat the process.

Assuming r is even, we calculate $a^{r/2}$. There is one more condition that must be satisfied. If

$$a^{r/2} \equiv -1 \pmod{N},$$

then the algorithm once again fails to produce useful factors, because adding 1 to it would give us a multiple of N , yielding a useless trivial factor when we run our checks. If that happens, we must restart with a different choice of a .

When these conditions are satisfied, something remarkable happens. Recall that

$$a^r \equiv 1 \pmod{N}.$$

Using the algebraic identity $x^2 - y^2 = (x - y)(x + y)$, we can write

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1).$$

This expression contains information about the factors of N .

We can extract that information by computing

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N).$$

These values often reveal non-trivial factors of N .

If we zoom out and look at the algorithm from a black-box perspective, it looks something like this:

Choose a $\longrightarrow$ Construct Periodic Function $\longrightarrow$ Find Period r $\longrightarrow$ Recover Factors

The most important observation here is that only one of these steps requires a quantum computer. Everything else can be performed using ordinary classical computation. In fact, the sole purpose of the quantum part of Shor's Algorithm is to find the period efficiently. Once the period is known, the remaining calculations are classical and comparatively straightforward. This is why the period-finding step is often considered the heart of Shor's Algorithm.

4 Quantum Fourier Transform

At this point we know that the most important part of Shor's Algorithm is finding the period r . The natural question is:

How does a quantum computer find this period?

This is where the Quantum Fourier Transform (QFT) enters the picture.

The QFT is one of the most important tools in quantum computing. In fact, several famous quantum algorithms make use of it in one form or another. Before understanding the quantum version, let's first build some intuition.

4.1 A Fourier Transform Intuition

Imagine that someone plays a musical note. To our ears it sounds like a single sound, but in reality that sound can be viewed as a combination of different frequencies. A Fourier Transform is a mathematical tool that takes a complex signal and tells us which individual frequencies are present inside it.

For our purposes, the most important thing to remember is:

Fourier Transforms are very good at detecting periodic patterns.

And what is the main thing we are looking for in Shor's Algorithm?

The Period

This is exactly why the QFT is useful.

The Quantum Fourier Transform performs a similar task, but instead of acting on ordinary acoustic signals, it acts on quantum states.

4.2 The Simplest Example

Believe it or not, you have already seen the simplest version of a QFT. Consider the Hadamard gate acting on a single qubit.

$$\begin{aligned}|0\rangle &\longrightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}} \\ |1\rangle &\longrightarrow \frac{|0\rangle - |1\rangle}{\sqrt{2}}\end{aligned}$$

The corresponding circuit is

$$|x\rangle \longrightarrow \boxed{H}$$

For a single qubit, the QFT is essentially equivalent to a Hadamard transform.

Of course, Shor's Algorithm works with many qubits, not just one. As the number of qubits increases, the QFT becomes more sophisticated, but the underlying idea remains the same:

Take a state containing periodic information and transform it into a state where that period becomes easier to observe.

4.3 How Does QFT Reveal a Period?

Suppose we have a quantum state whose amplitudes are concentrated at regular intervals. For example, imagine that measurements are likely to occur at

$$0, 4, 8, 12, 16, \dots$$

Notice that these values are separated by a period of 4.

Before applying QFT, this periodicity is hidden inside the state amplitudes. After applying QFT, the periodic structure gets converted into sharp peaks at locations related to the period.

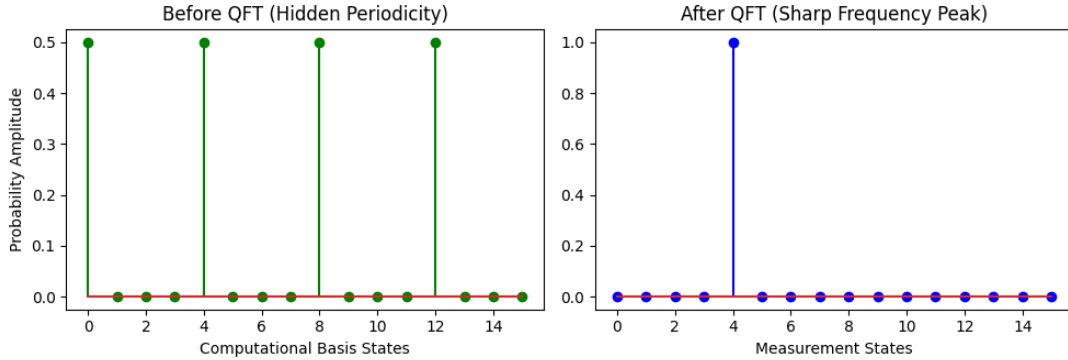


Figure 3: The Quantum Fourier Transform collapses distributed periodic amplitudes into a sharp, measurable frequency peak.

This is the key idea behind period finding. Instead of looking for the period directly, we transform the quantum state and observe where these peaks appear. From those peaks, we can recover the period.

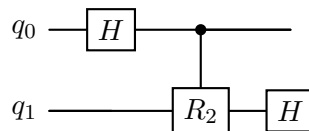
In a sense, the QFT converts period information into measurement information. This conversion is what gives Shor's Algorithm its power.

4.4 The QFT Circuit

The full QFT circuit consists of two basic building blocks:

- Hadamard gates.
- Controlled phase rotation gates.

For a two-qubit system, the circuit looks like:



where R_2 is a phase rotation gate.

As more qubits are added, more controlled phase rotations are introduced. Although the circuit may look complicated, every gate is working toward the same goal:

Expose the hidden periodic structure of the quantum state.

4.5 QFT Inside Shor's Algorithm

Choose $a \rightarrow f(x) = a^x \bmod N \rightarrow$ Create Quantum Superposition $\rightarrow$ Apply QFT $\rightarrow$
Recover Period $r \rightarrow$ Recover Factors

Notice that the QFT is only responsible for one task: finding the period. Once the period is known, the remaining calculations are classical. This is why people often say that the real quantum magic of Shor's Algorithm lies inside the Quantum Fourier Transform.

4.6 Qiskit Implementation of QFT

The easiest way to visualize a QFT circuit is with Qiskit.

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import QFT

qc = QuantumCircuit(3)
qc.append(QFT(3), [0, 1, 2])
print(qc.draw())
```

This code sets up a simple 3-qubit circuit and appends the pre-built QFT block from Qiskit. Printing the circuit lets us check the layout of Hadamard and phase gates, but there is no need to memorize it. The important thing is that it converts hidden periods into clear spikes we can measure.

5 Shor's Algorithm Step by Step

At this point we have all the necessary ingredients required to understand Shor's Algorithm. We know that the ultimate goal is factorization, and we also know that the quantum computer is primarily responsible for finding the period of a carefully constructed function. Let's now walk through the algorithm from start to finish using a concrete example.

Suppose we wish to factor $N = 15$. The first step is to choose a random integer a such that $1 < a < N$. For this example, let's choose $a = 2$.

We now construct the function $f(x) = 2^x \bmod 15$. Evaluating this function for a few values of x gives:

x	$2^x \bmod 15$
0	1
1	2
2	4
3	8
4	1

Notice that the sequence repeats after every four values. Therefore, $r = 4$.

The quantum computer is responsible for discovering this period efficiently. Once the period has been obtained, we calculate $a^{r/2}$. Substituting our values:

$$2^{4/2} = 4.$$

We then compute:

$$\gcd(4 - 1, 15) = \gcd(3, 15) = 3 \quad \text{and} \quad \gcd(4 + 1, 15) = \gcd(5, 15) = 5.$$

We have successfully recovered the factors:

$$15 = 3 \times 5.$$

This small example captures the essence of the entire algorithm. The difficult part is not computing the gcds. The difficult part is discovering the period r , which is exactly where the quantum advantage comes from.

6 Implementing Shor's Algorithm

Now that we have spent quite some time understanding the theory behind Shor's Algorithm, it is finally time to see it in action.

Before we begin, there is one thing I want to make clear. Implementing Shor's Algorithm completely from scratch is not a trivial task. As we have already seen, the algorithm involves several concepts such as modular arithmetic, period finding, quantum superposition, and the Quantum Fourier Transform.

Because of this, most people do not implement every component from scratch. Modern quantum computing frameworks such as Qiskit and PennyLane already provide many of the building blocks required for constructing the algorithm.

This allows us to focus on understanding how the algorithm works rather than spending hundreds of lines implementing every mathematical detail ourselves.

Before looking at the code, let's quickly recall what is happening behind the scenes. From a high-level perspective, the implementation follows the same steps that we discussed in the theory section:

Choose $a \longrightarrow f(x) = a^x \bmod N \longrightarrow$ Find Period $r \longrightarrow$ Recover Factors

The interesting thing is that only one of these steps is genuinely quantum. The period-finding stage is where the quantum computer does the heavy lifting. The remaining calculations, such as computing gcds and recovering factors, are classical computations.

As you go through the implementation, I would encourage you not to think of the code as a collection of commands that magically produce factors. Instead, try to identify where each theoretical concept appears in the implementation.

For example:

- Where is the periodic function being constructed?
- Where is the quantum state being prepared?
- Where is the Quantum Fourier Transform being applied?
- Where is the period extracted?
- Where are the factors finally recovered?

If you can answer these questions while reading the code, then you have truly understood the algorithm rather than simply running it.

Let's now build the implementation step by step.

6.1 Choosing the Number to Factor

The first thing we need to do is decide which number we want to factor. For demonstration purposes, we will use a relatively small number. Real-world applications involve much larger integers, but starting with a small example makes it easier to understand what is happening.

In our case, we choose $N = 15$. The goal of the algorithm is to recover the factors of this number. We also choose $a = 2$, which will later be used to construct the periodic function.

```
N = 15
a = 2
```

Here we just set our target number $N = 15$ and choose our base value $a = 2$ before starting the process.

6.2 Constructing the Periodic Function

The heart of Shor's Algorithm is the function $f(x) = a^x \bmod N$. For our example, this becomes $f(x) = 2^x \bmod 15$. Let's evaluate this function for a few values of x .

```
N = 15
a = 2
for x in range(10):
    print(x, pow(a, x, N))
```

This simple loop lets us look at the first few values of the function. Notice that we use Python's built-in 'pow(a, x, N)' instead of calculating a massive exponent first. This keeps the numbers small and the calculation very fast.

The output is:

```
0 1
1 2
2 4
3 8
4 1
5 2
6 4
7 8
...
```

Notice that the sequence starts repeating after every four values. This means that the period is $r = 4$. Of course, for very large values of N , we do not know this period in advance. Discovering it efficiently is exactly the task assigned to the quantum computer.

6.3 Preparing the Quantum State

Before the quantum computer can search for the period, it must first prepare a quantum state that contains information about many possible values of x . This is where quantum superposition enters the picture.

Instead of evaluating the function for one value of x at a time, the quantum computer creates a superposition of many possible inputs. Conceptually, if we looked at just the input register, it would look something like:

$$\frac{1}{\sqrt{2^m}} \sum_{x=0}^{2^m-1} |x\rangle$$

where m represents the number of working qubits. To make this work completely, the quantum computer actually sets up two quantum registers working in tandem. The first register stores the inputs $|x\rangle$ in superposition, and the second register calculates and holds the matching function output $|a^x \bmod N\rangle$.

When the function is calculated across all options simultaneously, it builds an entangled multi-register state that looks like this:

$$\frac{1}{\sqrt{2^m}} \sum_{x=0}^{2^m-1} |x\rangle |a^x \bmod N\rangle$$

The exact implementation details are not important for our current discussion. The key idea is that because our function values repeat, matching elements in the second register naturally link together inputs in the first register that are exactly spaced out by our hidden period r . This is one of the ingredients that eventually leads to the quantum speedup.

6.4 Applying the Quantum Fourier Transform

At this stage, information about the period is hidden inside the quantum state. Unfortunately, it is not directly observable. This is where the Quantum Fourier Transform (QFT) becomes useful. As discussed earlier, the QFT is extremely good at revealing periodic patterns.

Qiskit already provides an implementation of the QFT circuit.

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import QFT

qc = QuantumCircuit(3)
qc.append(QFT(3), [0, 1, 2])
print(qc.draw())
```

This sets up a three-qubit circuit and appends a pre-built QFT block. The QFT adjusts the internal phases of our state, causing incorrect paths to cancel out and leaving sharp peaks on our true frequencies.

The QFT transforms a state containing hidden periodic information into a state whose measurement outcomes are strongly related to the period. This is the most important quantum step of the entire algorithm.

6.5 Extracting the Period

Once the QFT has been applied, we perform a measurement. The measurement does not directly tell us the period. Instead, it provides a machine value y that tells us where a sharp spike happened. Mathematically, this value gives us a fractional estimate matching the relationship:

$$\frac{y}{2^m} \approx \frac{s}{r}$$

where 2^m is our total qubit capacity and s is just a random integer between 0 and r .

To extract the true denominator r out of this fractional relationship, we pass our measurement value through a quick classical script called the **Continued Fractions Algorithm**. This routine untangles the fraction and strips away s to successfully isolate our period.

For our example, the final expected result is:

```
Period r = 4
```

At this point, the difficult quantum part of the algorithm is essentially complete. The remaining calculations are classical.

6.6 Recovering the Factors

Now that we know the period, recovering the factors is surprisingly straightforward. Recall that $r = 4$. We first compute $a^{r/2}$. Substituting our values:

$$2^{4/2} = 4.$$

We then compute $\gcd(4 - 1, 15)$ and $\gcd(4 + 1, 15)$. The following code performs these calculations:

```
from math import gcd

N = 15
a = 2
r = 4

factor1 = gcd(pow(a, r//2) - 1, N)
factor2 = gcd(pow(a, r//2) + 1, N)
print(factor1, factor2)
```

We use Python's built-in 'gcd' function to calculate the greatest common divisor. By using our period $r = 4$, these simple equations instantly reveal the factors without any brute-force guessing.

The output is:

```
3 5
```

which gives us:

$$15 = 3 \times 5$$

We have successfully recovered the factors.

6.7 Complete Workflow Summary

$N \longrightarrow f(x) = a^x \bmod N \longrightarrow \text{Quantum Superposition} \longrightarrow \text{Quantum Fourier Transform} \longrightarrow$
Period $r \longrightarrow \text{Factors of } N$

Even though the mathematics behind Shor's Algorithm can become quite sophisticated, the overall idea is surprisingly elegant. Rather than attacking factorization directly, the algorithm transforms it into a period-finding problem. The Quantum Fourier Transform then helps reveal that period, allowing us to recover the factors using classical computation.

7 Limitations of Current Quantum Hardware

At this point it might seem like modern quantum computers should already be breaking RSA encryption everywhere. Unfortunately, the reality is much more complicated.

While Shor's Algorithm is theoretically capable of factoring very large integers efficiently, today's quantum computers are still relatively small and noisy. Running Shor's Algorithm on cryptographically relevant numbers would require a significantly larger number of qubits and much lower error rates than what current hardware can reliably provide.

As a result, most demonstrations of Shor's Algorithm today are performed on small numbers such as 15, 21, 35 or on quantum simulators. This should not be interpreted as a weakness of the algorithm itself. Rather, it reflects the current state of quantum hardware.

The situation is somewhat similar to the early days of classical computing. The theoretical ideas already existed long before the hardware became powerful enough to fully exploit them.

8 Final Remarks

Finally, You have just completed one of the most famous algorithms in all of quantum computing.

When people first hear about Shor's Algorithm, they often focus on the fact that it can factor large numbers efficiently. While that is certainly impressive, the real beauty of the algorithm lies in the idea behind it.

Instead of attacking a difficult problem directly, Shor's Algorithm transforms it into a completely different problem — one that a quantum computer happens to be very good at solving.

Throughout this chapter, we started with the problem of factorization, reduced it to period finding, explored the role of the Quantum Fourier Transform, and finally saw how all the pieces fit together to form a complete algorithm.

Even though many mathematical details were simplified along the way, the core ideas remain exactly the same as those used in the full version of Shor's Algorithm.

Don't worry if some parts of this chapter felt challenging. Shor's Algorithm is one of those topics that becomes clearer every time you revisit it. The goal of this chapter was not to make you memorize every detail, but to help you build an intuition for one of the most important ideas in quantum computing.

And if you have made it this far, congratulations — you now understand more about Shor's Algorithm than most people who have only heard its name.

Thank you for reading this far. I genuinely appreciate it. If you have any questions, suggestions, or spot any mistakes, feel free to reach out.

9 Further Reading

If this chapter sparked your curiosity, here are some excellent resources to explore the topic in greater depth:

- Michael Nielsen and Isaac Chuang, *Quantum Computation and Quantum Information* ““
- IBM Quantum Learning
- Qiskit Documentation
- PennyLane Documentation
- Peter Shor’s Original Paper (1994) ““